
STATISTICS FOR TRADERS

Jonas Löffler

loefflerjonas6@gmail.com

v1.1 · September 20, 2026

Abstract

Most trading mistakes are statistics mistakes wearing a costume. This paper builds, from zero, the statistical instincts a trader needs before any backtest or strategy discussion makes sense: seeing every return as a draw from a distribution you never observe directly; describing data with numbers that survive fat tails; separating a strategy's hit rate from its expectancy — a 70%-win strategy can lose money, and we show one doing it; reading dependence in time, where the direction of returns is nearly unpredictable but their size clusters for weeks; treating base rates as the bar every market opinion must clear, with hard rules against small-sample cells; and quantifying the uncertainty of every estimate by simulation, with no formulas. Every figure is computed from freely available data with short, fully listed scripts, and the paper closes with a guided setup — Python, pandas, free data sources, project hygiene, git — and one complete end-to-end analysis the reader can run in an afternoon. A glossary of about forty terms and an annotated reading ladder complete the foundation on which the rest of this library, beginning with the backtesting paper, is built.

Keywords — Distributions, fat tails, kurtosis, skewness, expectancy, hit rate, R-multiples, base rates, conditional probability, autocorrelation, volatility clustering, stationarity, standard error, bootstrap, confidence interval, free data, Python, pandas, reproducibility

1 Introduction

1.1 Motivation and thesis

Nobody loses money in markets because they cannot compute a standard deviation. They lose money because of statistical *instincts* that are miscalibrated: trusting a pattern seen four times, judging a strategy by how often it wins, assuming the next year will resemble the last six months, treating a scary average as a safe one because the word "average" sounds tame. Each of these is a precise, well-studied statistical error — and each has a precise, learnable correction.

The thesis of this paper is that a small set of statistical habits, none requiring more than school arithmetic, does more for a trader's survival than any indicator or setup: think in *distributions* rather than in single outcomes; describe data with numbers that remain honest when the data is wild; score strategies by *expectancy*, never by win percentage; demand a *base rate* before believing any conditional claim; refuse to read meaning into cells with a handful of observations; and attach an uncertainty estimate to every number you compute, because the number you computed is not the truth — it is a *guess at the truth*, made from one sample.

None of this is decoration before "the real work" of finding strategies. It is the real work. The companion paper on backtesting shows in detail how researchers with good intentions and bad statistical hygiene manufacture edges out of noise; this paper builds the hygiene. Everything here is shown, not asserted: every figure is computed from free public data by a short script you can run yourself, and every number quoted in the text is printed by one of those scripts.

1.2 Scope, prerequisites, and intended reader

This paper sits in the Trading Library's *Quant Methods* track and assumes nothing: no statistics, no programming, no market experience beyond knowing what a price is. School math — percentages, averages, a square root — suffices throughout. Every technical term is explained at first use and collected in the glossary (Section 10). Intuition and pictures come before any formula, and the few formulas that appear are arithmetic, not algebra.

This paper is also, deliberately, the only place in the library with a guided software installation and full beginner code. Later papers assume the setup of Section 7 and keep code minimal; here, the complete path from an empty computer to a finished analysis is walked step by step.

After reading this paper, a reader can:

- explain why a series of returns should be pictured as draws from a hidden distribution, and why the normal curve is a poor model of that distribution for markets — with the historical evidence, not slogans;

- summarize a dataset with mean, median, dispersion, skew, and kurtosis, and say when each of these numbers misleads;
- compute the expectancy of a trading rule from its hit rate and payoffs, and explain with numbers why hit rate alone is meaningless;
- distinguish correlation from causation with market examples, and read an autocorrelation chart;
- build a conditional base-rate table from raw prices without look-ahead, and apply the minimum-sample rule that keeps small cells from becoming stories;
- attach a bootstrap confidence interval to any simple estimate, by simulation, with zero formalism;
- and set up a free, reproducible Python research environment, pull free data, and run one complete analysis end to end.

1.3 Data and reproducibility

All figures in this paper use freely available data — Yahoo Finance for prices, and (in Section 7) FRED for the volatility complex and optionsDX for end-of-day option chains — cached locally so the figures are stable and reproducible offline. Two figures are pure seeded simulations and use no market data at all; each figure states which it is. This paper deliberately keeps the citation apparatus light: it is meant to be self-contained, and its references section is a short annotated reading ladder of published books rather than an academic bibliography. *This is a research and educational document, not investment advice.*

1.4 Organization of this paper

The paper runs from seeing to doing. Section 2 establishes the central picture — returns as draws from a machine you never see — and shows on a century of S&P 500 data how badly the normal curve fits reality. Section 3 builds the honest summary numbers (mean versus median, dispersion, skew, kurtosis) on that same dataset, and culminates in the most consequential distinction of the paper: expectancy versus hit rate, with a simulated 70%-win strategy that loses money. Section 4 adds time: correlation versus causation, the autocorrelation of returns versus their magnitudes, and what stationarity means and why markets do not have it. Section 5 turns to base rates and conditional probabilities — how to compute a conditional table from raw data, the minimum-cell rule ($n < 15$ is a story, not a statistic), and recency bias re-read as non-stationarity. Section 6 attaches uncertainty to every estimate via the bootstrap, entirely by simulation. Section 7 is the library's only guided tooling section: Python, pandas, matplotlib, free data sources, project hygiene, and git in ten commands. Section 8 puts everything together in one fully listed end-to-end analysis. Section 9 collects common misconceptions, Section 10 the glossary, and the

references close with a reading ladder and a pointer to the backtesting paper, for which this paper is the stated prerequisite.

2 Thinking in distributions

2.1 The machine you never see

Imagine the market as a machine that, every trading day, prints one number: that day's *return* — the percentage change of the price from yesterday's close to today's. You never get to open the machine. You never learn its true settings. All you ever see is the strip of numbers it has printed so far.

This is the single most useful mental picture in all of quantitative trading. The strip of past returns is not "what the market does"; it is a *sample* — one finite draw of outputs — from a hidden process that generates them. Statistics is the discipline of reasoning backward from the strip to the machine: what settings are *consistent* with what we have seen, and how confident can we be? Every honest statement about markets is a statement about the machine, qualified by the fact that we only hold a strip.

Three consequences follow immediately, and the rest of this paper unpacks them:

- **The strip could have come out differently.** If the machine has any randomness in it — and market returns behave as if it has a great deal — then rerunning history would print a different strip. Any number you compute from the strip (an average, a win rate, a "pattern") would come out differently too. That variability is *estimation error*, and Section 6 shows how to measure it.
- **The machine's settings may drift.** Nothing guarantees that the machine of 2008 is the machine of 2026. This is *non-stationarity*, the subject of Section 4.3, and it is the deepest reason market statistics is harder than textbook statistics.
- **Rare outputs are still outputs.** A day the machine has printed only once in a century is part of its repertoire. Whether your risk survives that day is not a detail; for leveraged traders it is usually the whole question.

2.2 The histogram: a picture of the strip

The first tool for looking at the strip is a *histogram*: chop the range of possible returns into small bins (say, quarter-percent wide), count how many days fall into each bin, and draw the counts as bars. Tall bars where returns are common, short bars where they are rare. A histogram is our best *estimate of the shape* of the machine's output distribution — imperfect, because it is built from a finite strip, but honest, because it shows every day we have.

The most famous idealized shape is the *normal distribution* (the "bell curve"): symmetric, thin-tailed, fully described by just two numbers — its center (the *mean*) and its width (the *standard deviation*, a measure of typical spread that we build properly in Section 3.2). The normal curve is beloved because it is mathematically convenient and because it genuinely describes many natural quantities. The question a trader must ask is not "is the normal curve elegant?" but "does the machine that prints returns actually behave like this?" That question has an empirical answer.

2.3 Fat tails, shown rather than asserted

Figure 1 takes every daily return of the S&P 500 available from Yahoo Finance — 24,794 trading days from 1927-12-30 to 2026-09-17 — and plots the histogram together with the normal curve that has *the same mean and the same standard deviation* as the data. In other words, the normal curve is given the best possible chance: it is fitted to this exact dataset. The vertical axis is logarithmic (each gridline is 10× the one below), which is what keeps single-day bins in the far tails visible at all; on an ordinary linear axis they would be invisible specks.

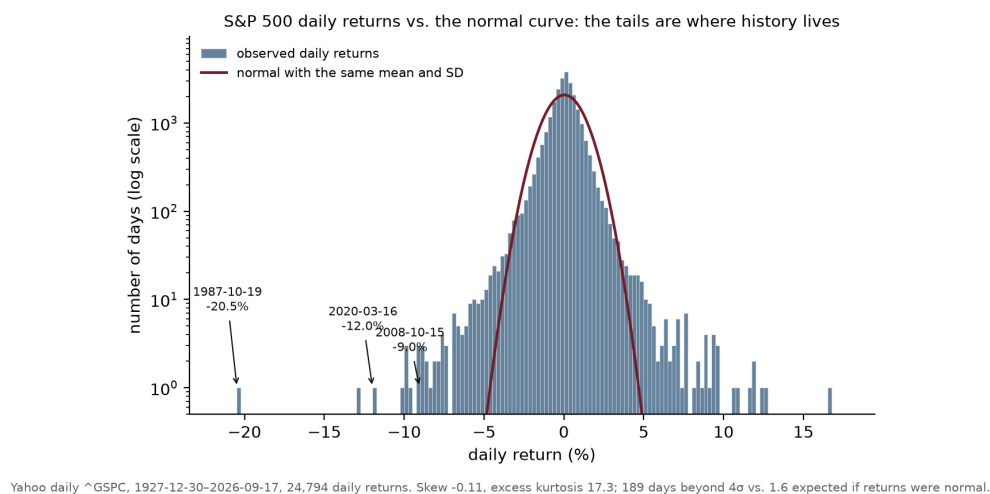


Figure 1. Histogram of 24,794 daily S&P 500 returns, 1927–2026, on a logarithmic count axis, with the best-fitting normal curve — same mean, same standard deviation — overlaid. The crash of 1987-10-19 at -20.5%, the worst day of October 2008 at -9.0%, and the worst day of March 2020 at -12.0% sit far outside the region where the normal curve has effectively collapsed to zero. Reproduce with `figures/fig_return_histogram.py`.

Read the figure from the center outward. Near zero, the fit looks tolerable — in fact the data has *more* small, quiet days than the normal curve predicts (the observed peak pokes above the red line). But the tails are a different world. The red curve dives to effectively zero beyond about $\pm 5\%$, while the blue bars march on: -7%, -9%, -12%, and one lonely bar at -20.5% — Monday, October 19, 1987. The worst day of October 2008 (-9.0%) and the worst day of March 2020 (-12.0%) are marked as well. These are not smudges or data errors. They are days that happened, and the normal curve says they essentially cannot.

How badly does the normal curve fail? Make it precise with the *z-score* — a return expressed in units of standard deviations from the mean. Over this sample the standard deviation of daily returns is 1.19%, so 1987-10-19 was a -17.2σ day, 2020-03-16 a -10.1σ day, 2008-10-15 a -7.6σ day. Under a normal distribution, a day beyond $\pm 4\sigma$ should occur roughly once every 15,800 trading days — about 1.6 times in our whole 99-year sample. The actual count is 189 — about 120 times too many. And for the 1987 print, the normal probability of a single -17σ day is on the order of 1 in 10^{65} ; for scale, only about 5×10^{12} days have elapsed since the Big Bang. A model that assigns cosmically impossible odds to an event that occurred within living memory is not a slightly imperfect model. It is the wrong model for the tails — and for a trader, the tails are where ruin lives.

This property — far more extreme days than the normal curve allows — is called *fat tails* (or *heavy tails*), and it is one of the most robust findings in all of empirical finance, documented since Mandelbrot's cotton-price studies in the 1960s and confirmed on essentially every liquid market since. In Section 3.3 we will attach the standard summary numbers (skewness, kurtosis) to it.

2.4 What fat tails change in practice

Almost everything, but three consequences matter most:

1. **"X-sigma event" language malfunctions.** Headlines calling a move a "10-sigma event" implicitly use the normal yardstick, under which such an event is impossible — so the correct conclusion is never "how unlucky!" but "the yardstick is wrong." Fat-tailed machines produce multi-sigma days as a matter of routine; Figure 1 shows 189 of them beyond 4σ .
Risk lives in the tail, not the middle. Two strategies can have identical average returns and identical standard deviations while one of them carries a rare -20% day and the other does not. Summary numbers that only describe the middle of the distribution — and the standard deviation largely does — can make those two strategies look like twins. Section 3 builds the vocabulary to tell them apart.
2. **Averages need far more data than intuition expects.** In a thin-tailed world, a few hundred observations pin down an average nicely. In a fat-tailed world, a single day can move a multi-year average — one 1987 outweighs two ordinary years of drift. Estimates converge slowly, and Section 6 shows how to measure how slowly.

The one-sentence version. The market prints from a machine you never see; the strip you hold is one sample; and the machine's tails are far heavier than the bell curve — so every method in this library must be judged by how it behaves when the tail arrives.

3 Describing data honestly

A distribution is a shape; day to day we work with a handful of summary numbers. This section builds them in order — center, spread, shape — on the same S&P 500 dataset as Figure 1, and then applies them to the number traders care about most: what a trading rule earns per trade.

3.1 Center: mean versus median

The *mean* (the ordinary average: add everything, divide by the count) of the 24,794 daily returns is +0.032% per day — small, positive, and the compounding engine behind a century of equity returns. The *median* — the middle value when all returns are sorted, half the days above it, half below — tells you what a *typical* day looks like.

The two answer different questions. The mean is what you *accumulate per day on average* if you hold through everything — every point contributes, so a -20% day drags the mean with its full weight. The median is *the day in the middle* — it barely notices whether the worst day was -7% or -20%, because either way that day just sits at the bottom of the sorted pile. Under symmetric, well-behaved data, mean and median agree and the distinction is pedantic. Under skewed or fat-tailed data they diverge, and the divergence is diagnostic: a strategy whose mean is far below its median is quietly telling you that its losses are rare but violent — the classic profile of strategies that "always win" until they don't. Whenever you see an average, ask for the median next to it; when the two disagree, the interesting part of the distribution is the part that made them disagree.

3.2 Spread: the standard deviation and its limits

The center says nothing about how wildly outcomes vary around it. The workhorse measure of spread is the *standard deviation*: loosely, the typical distance of a data point from the mean. (Mechanically: take each point's distance from the mean, square the distances, average the squares — that average is the *variance* — and take the square root to get back to ordinary units. For a *sample* estimate the squares are divided by $n - 1$ rather than n — Bessel's correction for the degree of freedom spent estimating the mean — which is the default in *numpy* and *pandas* and matters on small samples, where dividing by n understates the spread.) For our daily returns it is 1.19%: a "typical" S&P 500 day deviates about one percent from its average. In markets, standard deviation wears a second name: *volatility*. When volatility is quoted annualized (as in "VIX at 20"), it has been scaled up from daily units by the square root of the number of trading days in a year — a convention, not a mystery.

Two honest warnings. First, because deviations are *squared*, the standard deviation is hypersensitive to outliers — one 1987 contributes as much to the variance as roughly three hundred ordinary $\pm 1\%$ days. Second, and more important after Section 2: two distributions with the *same* standard deviation can carry entirely different tail risk. The standard deviation is a fine

everyday ruler and the industry's default, but it is a middle-of-the-distribution number; it must never be your only risk number.

3.3 Shape: skewness and kurtosis

Two more summary numbers describe the *shape* of the histogram, and both are best read as alarms rather than precise measurements:

- **Skewness** measures asymmetry: zero for a symmetric shape, negative when the long tail points left (rare but violent losses), positive when it points right (rare but violent gains). Our full-history sample has a skewness of -0.11 — mildly negative. That may look surprisingly tame given 1987, and the reason is instructive: the biggest *up* days in history (several beyond $+10\%$, visible on the right of Figure 1) sit inside the same tails and pull the number back toward zero. Skewness estimated from fat-tailed data is itself fragile — a single day can move it — so treat its sign as a hint, not its second decimal as truth.
- **Kurtosis** measures tail weight — how much of the distribution's variability comes from rare, extreme outcomes rather than everyday wiggle. It is usually quoted as *excess kurtosis*: the value over and above the normal distribution's, so that the normal scores exactly 0. Our sample's excess kurtosis is 17.3. There is no need to memorize the formula; the calibration to carry is that 0 means "normal-like tails," anything above about 1–2 means "noticeably fat," and 17 means "the normal model is not merely imprecise but category-wrong." Figure 1 is what an excess kurtosis of 17 looks like.

Together with mean and standard deviation, these four numbers are the standard first summary of any return series — and the honest habit is to compute all four *plus the median*, every time, before believing any of them individually.

3.4 Why averages lie under fat tails

Put Section 3.1–Section 3.3 together and a practical warning drops out. An average computed from a fat-tailed series is dominated by its rare extremes — which means the sample you have may simply *not contain* the extreme that dominates the true machine. A strategy backtested over five calm years shows you its behavior *conditional on no earthquake*; its true mean includes the earthquake. This is why a short, smooth track record is weak evidence, why Section 6 insists on error bars, and why the backtesting paper insists that the *distribution of trades* — not the equity curve — is the object a backtest must be judged on. Averages are not wrong; they are merely answers to the question "what happened per day *in this sample*," and under fat tails that question and "what will happen per day" can be far apart.

3.5 Expectancy versus hit rate: the trader's version of the mean

Now aim the same machinery at the number that actually decides whether a trading rule makes money. Traders love to quote the *hit rate* (or win rate): the percentage of trades that close profitable. It is the most seductive number in trading and, alone, it is meaningless.

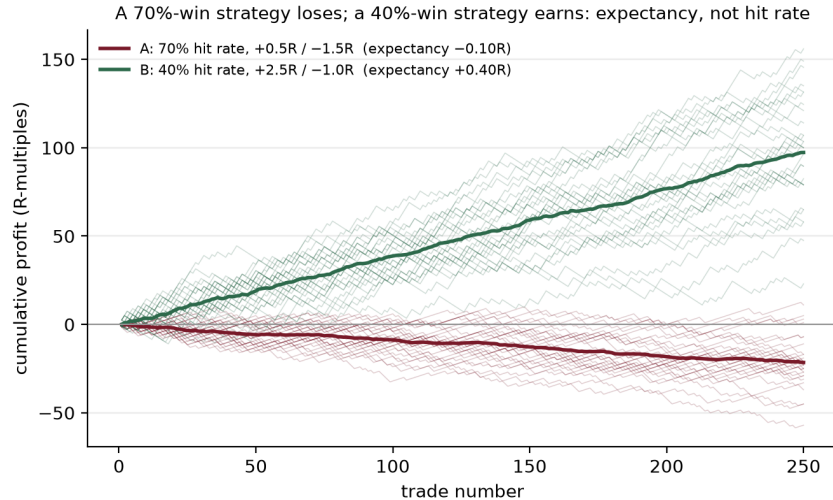
What decides profitability is *expectancy*: the average profit per trade — the mean of the trade-outcome distribution — which combines the hit rate with the *payoff*: how much a winner wins versus how much a loser loses. It is convenient to measure outcomes in *R-multiples*: R is the amount risked on the trade (the distance to your stop, in money), and every outcome is expressed as a multiple of it, so "+2.5R" means the trade made two and a half times what it risked. In R-units, the arithmetic is school-level:

Expectancy per trade = (hit rate × average win) – (loss rate × average loss).

Work one example with real numbers, twice.

- **Strategy A** wins 70% of the time, but takes profits fast and lets losses run: average win +0.5R, average loss –1.5R. Per 100 trades: $70 \times 0.5R = +35R$ won, $30 \times 1.5R = -45R$ lost. Net: –10R per 100 trades, an expectancy of –0.10R. Seventy percent winners, and it bleeds money — indefinitely, by construction.
- **Strategy B** wins only 40% of the time, but wins pay +2.5R against –1.0R losses. Per 100 trades: $40 \times 2.5R = +100R$ won, $60 \times 1.0R = -60R$ lost. Net: +40R per 100 trades, an expectancy of +0.40R. It is wrong more often than it is right, and it compounds wealth.

Figure 2 lets both strategies actually trade, as a seeded simulation (no market data): 30 independent 250-trade sequences per strategy. The realized averages match the arithmetic — –0.086R and +0.389R per trade across all 7,500 simulated trades of each — and every green path family climbs while the red family sinks.



Seeded simulation (numpy seed 42), 30 paths of 250 trades per strategy; thin lines are single paths, bold lines the average path. No market data.

Figure 2. Thirty simulated 250-trade equity paths for each of two strategies, in cumulative R-multiples. Strategy A wins 70% of trades with payoffs +0.5R against -1.5R and has expectancy -0.10R per trade; strategy B wins 40% with payoffs +2.5R against -1.0R and has expectancy +0.40R. The 70%-winner loses steadily; the 40%-winner compounds. Seeded simulation, no market data. Reproduce with `figures/fig_expectancy.py`.

Two further lessons hide in the thin lines. First, 20% of the losing strategy's paths are in profit after 50 trades — a fifth of the people trading a mathematically losing rule would feel vindicated after two months, purely by chance. This is why "it's been working for me" is not evidence; Section 6 makes the point quantitative. Second, the psychological trap runs the other way too: strategy B spends most of its time *losing individual trades*. A 40% hit rate means routine strings of four, five, six consecutive losses; abandoning a positive-expectancy rule mid-losing-streak is how sound systems die in live hands.

The vocabulary of this section — expectancy, hit rate, payoff, R-multiples — is used throughout the rest of the library exactly as defined here, and the backtesting paper evaluates every strategy in these units rather than by equity curves.

4 Dependence and time

So far each day was a separate draw. But returns arrive in a sequence, and the sequence itself carries structure. This section covers the three ideas that keep sequence-reasoning honest: correlation (and its abuse), autocorrelation (what actually repeats), and stationarity (what "the past resembles the future" would even mean).

4.1 Correlation is not causation — two market examples

Correlation measures how two series move together, on a scale from -1 (perfect opposition) through 0 (no linear relationship) to $+1$ (perfect lockstep). It is a purely descriptive number: it counts co-movement and knows nothing about *why*. Two market examples show the two classic failure modes.

Example 1: VIX and the S&P 500 — co-movement without a one-way cause. Daily changes in the VIX index (a measure of the expected volatility implied by S&P 500 option prices) are strongly negatively correlated with S&P 500 returns: on most days when stocks fall hard, the VIX jumps. It is tempting to read a causal arrow: "rising VIX causes selling," or the reverse. But both series respond jointly to the same underlying events — news, deleveraging, changes in risk appetite — and each also feeds back into the other through hedging flows. The correlation is real, stable, and useful for *description*; what it does not license is a story in which one series is a lever that drives the other. Whenever two market series co-move, the boring hypothesis — both are downstream of a common cause — must be eliminated before any causal claim, and it usually cannot be.

Example 2: butter in Bangladesh — correlation manufactured by search. In a famous demonstration, David Leinweber searched a large international dataset for whatever best "explained" annual S&P 500 returns and found that butter production in Bangladesh fit superbly — with dairy and sheep-population variants pushing the in-sample fit higher still. The point of the joke is deadly serious: if you scan enough candidate series, *some* of them will correlate strongly with your target by pure chance, and the more you scan the more impressive the best coincidence will look. (The "Super Bowl indicator" belongs to the same family.) A correlation you *found by searching* is an entirely different object from a correlation you *predicted in advance* — the same distinction between exploratory and confirmatory analysis on which the backtesting paper builds its whole methodology.

The discipline that follows from both examples fits in one line: a correlation supports a causal story only when the story was stated *before* looking, predicts a *direction* (sign), and survives on data that was not used to find it.

4.2 Autocorrelation: what actually repeats

The most natural question about the return sequence is whether it remembers itself: does today's return predict tomorrow's? The tool is *autocorrelation* — the correlation of a series with a delayed copy of itself. The delay is called the *lag*: lag 1 compares each day with the next day, lag 5 with the day a week later. And the trick that makes Figure 3 the most quietly important chart of this paper is to compute autocorrelation *twice*: once on the returns themselves (does the *direction* repeat?), and once on their absolute values (does the *size* of moves repeat, ignoring direction?).

The shaded band shows roughly how large an autocorrelation pure noise would produce in a sample of this size (3,770 daily returns, 2011–2026); bars inside the band are indistinguishable from nothing.

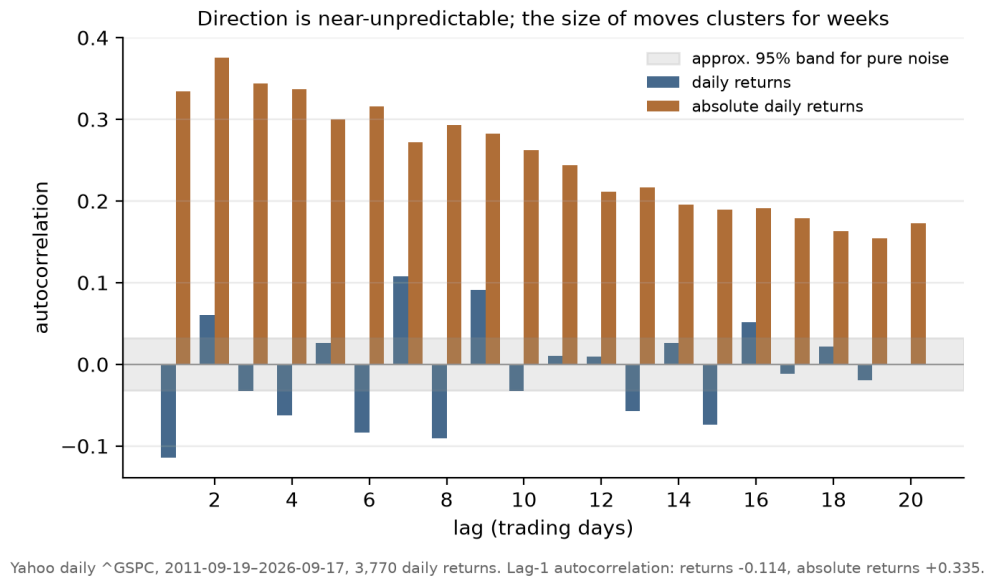


Figure 3. Autocorrelation of S&P 500 daily returns (blue) and of their absolute values (orange) for lags 1 to 20, on 3,770 daily returns 2011–2026, with an approximate 95% band for pure noise. Return autocorrelations are tiny — lag 1 is -0.114 — while absolute-return autocorrelations start at $+0.335$ and remain outside the noise band at every one of the 20 lags: direction barely repeats, size clusters for weeks. Reproduce with `figures/fig_autocorrelation.py`.

The two series could hardly behave more differently:

- **Returns (blue):** all bars are small. The largest, at lag 1, is -0.114 — mildly negative, meaning an up day is followed by a *very slightly* below-average day and vice versa. Honesty requires noting that 12 of the 20 lags do poke outside the noise band: the effects are statistically detectable in a sample this large. But detectable is not tradable — an autocorrelation of -0.11 is a whisper, and the backtesting paper shows precisely this whisper being traded (a 1-day reversal rule built on this very lag-1 effect) and dying under about 2.7 basis points of one-way transaction cost. Small-but-significant is a pattern you will meet constantly; the reflex it should trigger is "significant *and* large enough to survive costs?", never "significant, therefore edge."
- **Absolute returns (orange):** a different universe. Lag 1 is $+0.335$, and *all twenty* lags clear the noise band, decaying only slowly. Big-move days follow big-move days; quiet days follow quiet days — for weeks. This is *volatility clustering*, one of the most robust regularities in finance: the market's *direction* is nearly unpredictable, but its *temperature* is strongly persistent.

This asymmetry is the empirical heart of the whole volatility track of this library: forecasting *whether* tomorrow rises is close to hopeless; forecasting *how much* markets will move is a genuinely tractable problem, because persistence is right there in the data. It also explains a paradox from Figure 1: fat tails and clustering are linked, since extreme days arrive in storms (October 2008, March 2020) rather than sprinkled independently through time.

4.3 Stationarity: the fine print under every statistic

Every statistic in this paper — every mean, histogram, autocorrelation, base rate — silently assumes that the numbers being pooled came from *the same machine*. The formal name for that assumption is *stationarity*: the distribution generating the data does not change over time. Under stationarity, more data is always better, and the past is a fair sample of the future.

Markets fail this assumption, visibly and structurally. Figure 3 already proves a mild failure: volatility clustering *means* the distribution's width is different in different months — pooling March 2020 with August 2017 averages over two machines with very different settings. The deeper failures are slower: market structure changes (decimalization, high-frequency market making, the growth of index options and 0DTE trading), policy regimes change (zero rates versus 5% rates), and participants *adapt* — a pattern widely exploited tends to fade, because the exploiting is itself trading pressure. A chemistry experiment does not change its outcome because chemists published; markets do.

Three practical consequences, stated plainly:

1. **Every historical statistic is an average over regimes.** The unconditional numbers of Figures 1–4 blend crash years and calm years. That does not invalidate them — it defines what they are: long-run climatology, not tomorrow's weather.
2. **More history is not automatically better.** Extending a sample back another 30 years adds data from a machine increasingly unlike today's. There is a genuine trade-off between statistical comfort (large n) and relevance (recent regime), and Section 5.4 gives the honest way to live with it — show both windows, never let one silently replace the other.
3. **Nothing "holds" — things "have held."** The only defensible grammar for a market statistic is past tense plus a sample: "P(up day) *was* 54% over 2011–2026." The moment a sentence about markets drops its date range, it has claimed stationarity it cannot prove.

5 Base rates and conditional probabilities

5.1 The base rate: the bar every opinion must clear

The *base rate* of an event is its plain, unconditional frequency: out of all days in the sample, what fraction did the thing happen? It is the single most protective number in practical statistics, because every impressive-sounding claim must be measured *against* it — and almost never is.

Compute one. Over 3,571 evaluation days of S&P 500 history (2011–2026; the sample construction is in Section 5.2), the market closed up on 54.3% of days. That is the base rate of an up day in this era. Now hold it against a claim you will hear daily in trading media: "*Setup X just fired — historically, 58% of such days closed green!*" Without the base rate, 58% sounds like an edge. Against a 54% base rate it is a two-point-something improvement — which, as Section 6 will show, is well within estimation noise unless the setup fired hundreds of times. The base-rate reflex, in psychology literature since Kahneman and Tversky documented *base rate neglect*, translates to trading as one question asked relentlessly: "**...compared to what?**" A forecast, a pattern, or a paid signal service earns attention only for its *lift* — its accuracy *minus* the base rate a coin-flipping monkey with a calendar would achieve. A prediction with the base rate's accuracy is the base rate in an expensive costume.

5.2 Conditional probabilities: one pre-committed table

The natural refinement is the *conditional probability*: not "how often does the market close up?" but "how often does it close up *given* some condition visible beforehand?" — written $P(\text{up} \mid \text{condition})$. Computing conditional probabilities from raw data is mechanical: keep only the days where the condition held, and take the fraction of those days that closed up. The dangers are not mechanical, and the table below is built to demonstrate both of them.

Figure 4 shows *one* conditional table on the same S&P 500 sample, with every design decision made *before* computing — the same pre-commitment discipline the backtesting paper formalizes. The outcome is always "day t closes up." Every condition uses only information available at the close of day $t-1$ — yesterday's return sign, yesterday's close versus its own 200-day moving average (the average of the last 200 closing prices, a standard slow trend measure) — never anything from day t itself. Using information that was not yet knowable is *look-ahead bias*, the cardinal data sin, and the reason the first 199 days of the sample are dropped (a 200-day average needs 200 days of history). Two deliberately extreme rows — yesterday fell more than 3%, yesterday fell more than 5% — were pre-committed precisely to demonstrate the small-sample rule of Section 5.3. No other conditions were tried; what the table shows is reported as it fell.

Condition (known at close of day t-1)	n	P(day t up)	bootstrap 90% CI
unconditional	3,571	0.543	0.529 – 0.557
prior day up	1,940	0.525	0.506 – 0.543
prior day down	1,630	0.565	0.545 – 0.585
above 200-day average	3,010	0.546	0.531 – 0.560
at/below 200-day average	561	0.531	0.497 – 0.565
prior up & above 200d	1,682	0.524	0.504 – 0.543
prior up & below 200d	258	0.531	0.481 – 0.581
prior down & above 200d	1,327	0.573	0.550 – 0.595
prior down & below 200d	303	0.531	0.485 – 0.578
prior day $\leq -3\%$	35	0.486	0.343 – 0.629
prior day $\leq -5\%$	6	0.833	0.500 – 1.000

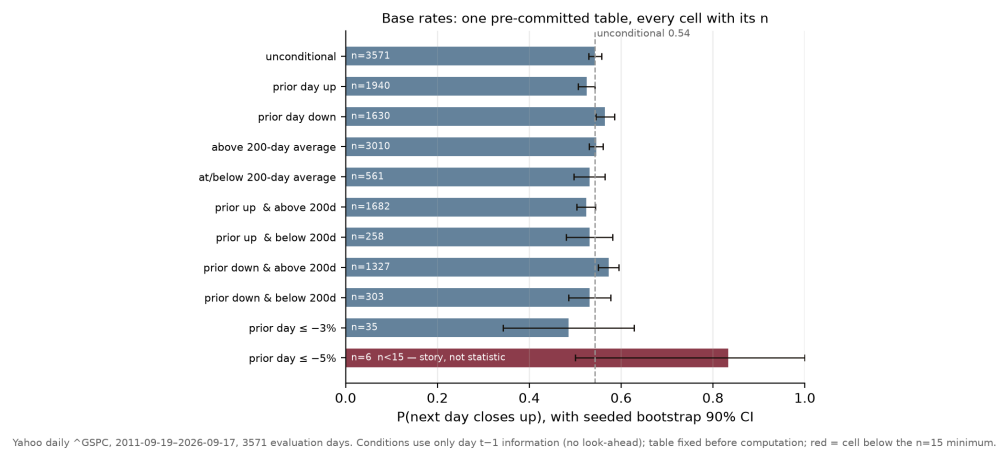


Figure 4. One pre-committed conditional base-rate table on 3,571 S&P 500 evaluation days, 2011–2026: probability that the next day closes up, unconditionally and conditional on the prior day's sign and on the close relative to its 200-day average, each cell with its sample size and a seeded bootstrap 90% confidence interval. The dashed line marks the unconditional rate of 0.543. The bottom cell — prior day down more than 5% — shows an apparent 83% up-rate on $n = 6$ and is flagged in red: below the $n = 15$ minimum it is a story, not a statistic. Reproduce with `figures/fig_base_rates.py`.

Read it honestly, because honest reading is the skill being taught:

- **The conditions barely matter.** Every credible cell sits within about three points of the 54.3% base rate. The strongest credible cell — *prior day down while above the 200-day average*, 57.3% on $n = 1,327$ — is a modest, real-looking tilt (it is the same short-horizon

reversal whisper as Figure 3's lag-1 bar, seen through a different lens; two views of one phenomenon, not two discoveries). Nobody retires on three points of conditional probability before costs.

- **This is what most conditioning looks like.** The flat table is not a failed figure; it *is* the result. The everyday conditions traders talk about most — up day, down day, above or below the big moving average — move next-day odds barely at all. Expect this outcome as the default whenever you condition on something popular, and treat any table where cells swing dramatically as a reason to check *n* before checking your excitement.

5.3 The small-sample trap: $n < 15$ is a story, not a statistic

Now the bottom row. *After a one-day drop of more than 5%, the market closed up 83% of the time.* An 83% up-rate! Against a 54% base rate! It is the most exciting number in the table — and it is built on **six days**. Five of six closed up. With samples this small, the outcome is essentially anecdote: had a single one of those six days gone the other way, the cell would read 67%; two, and it reads 50% — perfectly ordinary. The bootstrap interval in the table says the same thing formally: the data is consistent with anything from a coin flip to certainty.

This library therefore applies a hard editorial rule, adopted from its private research conventions and enforced in every paper: **a conditional cell with n below 15 may not be quoted as evidence for anything.** It is flagged (as in Figure 4, in red), reported for completeness, and treated as a *story* — a prompt for a future hypothesis, never support for a present one. Fifteen is not a magic constant; it is a deliberately blunt tripwire placed well inside the danger zone, cheap to obey and expensive to ignore. Cells between roughly 15 and 100 deserve suspicion in proportion; the row above ($n = 35$, and its wide interval spanning 0.34–0.63) shows why. The trap generalizes viciously, and it compounds with Section 4.1's search problem: the more conditions you stack ("down >2% *and* below the 200-day *and* a Friday *and* December..."), the smaller each cell gets and the wilder the percentages look — 100% and 0% cells bloom everywhere once n drops to 3. Fine-sounding multi-condition "setups" quoted with two-digit precision are, more often than not, $n < 10$ cells that nobody printed the n for. The two-part defense costs one line in any table: *always print n next to every percentage, and never believe a percentage whose n you have not seen.*

One more habit belongs here because small samples are where it matters most: the *direction check*. Before letting any statistic support a claim, verify that its sign actually points the way the claim says — that "crash days are followed by strength" is being supported by data showing strength, not by an impressive-looking number measuring the opposite. Stated so plainly it sounds unnecessary; in practice, excited pattern-hunters routinely absorb a striking number *as* support without checking its direction, and the private research behind this library includes one expensive incident of exactly that shape. Check the sign, then the size, then the n — in that order.

5.4 Recency bias is non-stationarity in disguise

Recency bias is the tendency to overweight the latest data — after three trending months, believing markets trend; after a volatile fortnight, believing volatility is the new normal. Classical statistics calls this a bias, and it is. But Section 4.3 complicates the sermon: because markets are non-stationary, recent data genuinely *is* sometimes more representative of the current machine than decade-old data. The trader who "overweights" the post-2020 regime is doing something defensible that a naive long-sample average is not.

So the failure is subtler than "ignoring history" or "chasing recency." Both windows are estimates of different things — the long window estimates climate, the short window estimates (noisily) the current weather — and the sin is *silently substituting one for the other*, in either direction. The honest protocol, used in this library's own session research, costs two lines:

1. **Report both windows, each with its n.** "P(up) = 54.3% over 15 years (n = 3,571); 58% over the last 60 sessions (n = 60)." Now the reader — including future you — sees the climate, the recent deviation, and exactly how little the recent number is based on.
2. **Let the short window color, never carry.** A recent shift is a hypothesis about regime change, to be confirmed by data that arrives after it was stated — not a licence to discard the long sample. If the shift is real, it will still be there next month, with a larger n; if it was noise, you just declined to bet your account on sixty coin flips. A short window's n is small *by construction*, so by Section 5.3's rule it is almost always a story — and the market rewards nobody for promoting stories to statistics ahead of schedule.

6 How sure are we? Standard errors by simulation

6.1 An estimate is not the truth

Every number computed so far — 54.3%, -0.114, +0.389R — is an *estimate*: a value calculated from one finite strip printed by the machine of Section 2.1. Rerun history and every one of them comes out slightly different. The *standard error* is the name for the typical size of that "slightly": how much the estimate would wobble across alternative strips. It is the difference between an estimate and a fact, made quantitative — and the habit of asking for it is what separates statistical thinking from number-quoting.

Textbooks derive standard-error formulas. We will not use a single one, because there is a method that needs no formulas, works for nearly any statistic, and *shows* you the uncertainty instead of asserting it.

6.2 The bootstrap: rerunning history from the sample you have

The idea, called the *bootstrap*, is almost embarrassingly direct. You cannot rerun the market to get more strips — but you have one strip, and you can generate *plausible alternative strips* from it: build a fake sample of the same size by drawing days at random from your real sample, **with replacement** (each draw takes any of the original days, so some days appear twice in the fake sample and others not at all — that is what makes each fake strip differ from the original). Compute your statistic on the fake strip. Now repeat ten thousand times — trivial for a computer — and look at the spread of the ten thousand results. That spread is a direct, visual answer to "how much does my estimate depend on the luck of my particular sample?" Chop off the bottom and top 5% of the ten thousand results and the range that remains is a *90% confidence interval*: the range of values your data cannot seriously distinguish from the one you computed. No formula was harmed; the entire method is "resample, recompute, repeat, look."

Every interval quoted in this paper was produced exactly this way, by seeded resampling inside the figure scripts (seeded meaning the random draws are fixed and reproducible — Section 7.3), and you can reread Figures 2 and 4 as bootstrap galleries.

6.3 Reading the intervals — three lessons from our own figures

Lesson 1: fifty trades know almost nothing. Take the first simulated path of the profitable strategy B (true expectancy +0.40R). After 50 trades its realized average happened to be +0.61R, and the bootstrap 90% interval around that estimate spans **+0.19R to +1.03R** — the data cannot tell a modest edge from a spectacular one. The losing strategy A's first path after 50 trades: −0.22R with an interval of **−0.46R to −0.02R**, barely excluding zero; and recall from Section 3.5 that a fifth of A's paths were *positive* at trade 50. Fifty trades — months of discretionary trading — cannot reliably distinguish a good strategy from a bad one.

Lesson 2: even 250 trades leave real doubt. After 250 trades, A's first path averages −0.084R with an interval of **−0.180R to +0.012R**: the upper end still touches zero — a full year of trading a rule that loses money *by construction*, and the data still cannot quite certify that it loses. Uncertainty shrinks with sample size, but slowly — as the *square root* of n , so halving the error bar costs *four times* the data. This is the quantitative engine behind every "be patient with evidence" sermon in this library, and the reason the backtesting paper treats short backtests as near-worthless.

Lesson 3: the interval is the antidote to the exciting cell. The 83% cell of Section 5.3 carries the interval **0.50 to 1.00** — the visual whisker in Figure 4 spans half the axis. The unconditional rate, with $n = 3,571$, carries **0.529 to 0.557** — a tight whisker you can actually lean on. Same table, same method, same market; the *only* difference is n . Once you have seen the two whiskers

side by side, "a percentage without an n" stops being information and starts being noise with confident formatting — which is exactly the reflex this section exists to install.

The one-sentence version. Every estimate deserves an error bar, the bootstrap gives you one with no mathematics beyond "resample and repeat," and most trading sample sizes are far too small to support the confidence with which they are quoted.

7 Tooling: a working research setup in an afternoon

Everything in this paper was computed with free tools on free data, and this section — the only guided-install section in the library — takes you from an empty computer to the same capability. Later papers assume this setup exists.

7.1 Installing Python, pandas, and matplotlib

Python is the standard language of quantitative research; *pandas* is its workhorse library for tabular data (think: a programmable spreadsheet built for time series); *matplotlib* draws the charts. Install order:

1. **Python.** Download the current version from python.org and run the installer. On Windows, tick "**Add Python to PATH**" on the first screen — it is the single checkbox that prevents most beginner misery. On macOS/Linux, the system usually has Python; `python3 --version` in a terminal confirms.
2. **A project folder with a virtual environment.** A *virtual environment* is a private copy of Python's package space, one per project, so projects can never break each other — professional habit, zero cost, day one:

```
mkdir trading-research
cd trading-research
python -m venv .venv
.venv\Scripts\activate      (Windows)
source .venv/bin/activate   (macOS/Linux)
```

The `(.venv)` prefix in your prompt means the environment is active; activate it whenever you work on the project.

1. **The libraries.** One line:

```
pip install pandas matplotlib yfinance jupyterlab
```

1. **Verify** (should print a recent pandas version and open a blank chart window's worth of nothing — i.e., no errors):

```
python -c "import pandas, matplotlib, yfinance; print(pandas.__version__)"
```

7.2 Free data: prices, the volatility complex, options

Three free sources cover essentially everything this library does, and none requires payment:

- **Prices — yfinance.** The yfinance package pulls Yahoo Finance's price history: decades of daily data for indices (^GSPC for the S&P 500, ^NDX for the Nasdaq-100), single stocks and ETFs. `yf.Ticker("^GSPC").history(start="2005-01-01")` returns a ready pandas table. This is the source behind every price figure in this paper.
- **The volatility complex — FRED.** The Federal Reserve's FRED database (fred.stlouisfed.org) republishes CBOE's volatility indices as clean daily series, downloadable as CSV from the site or directly by URL — <https://fred.stlouisfed.org/graph/fredgraph.csv?id=VIXCLS> fetches the VIX; `VXNCLS` (Nasdaq-100 vol) and the term-structure companions live under similar IDs. Free, no registration, and the standard source for the volatility track of this library.
- **Options end-of-day — optionsDX.** Historical option-chain data is the expensive dataset everywhere else; optionsdx.com provides end-of-day US option chains (strikes, expirations, prices, greeks) free with registration. When the options-track papers need chain data, this is the assumed source.

One discipline from day one: **cache what you download.** Save every fetched dataset to a local file (CSV is fine) and have your scripts read the file if it exists, fetching only when it does not — exactly the pattern in this paper's figure scripts. Your work stops depending on servers, rate limits, or a data source quietly revising history under you, and your figures become reproducible offline.

7.3 Project hygiene from day one

Five habits, each nearly free now and expensive to retrofit later:

- **A standard folder layout.** One shape serves every project in this library:

trading-research/	
data/raw/	downloaded files - never edited, ever
data/processed/	anything your code derived from raw
notebooks/	exploration (Jupyter)
scripts/	anything you will run twice
figures/	charts your scripts produce
results/	tables, logs, outputs

- **Raw data is read-only.** Nothing in `data/raw/` is ever opened in Excel and "fixed." Every transformation lives in code, so every derived number has a visible, rerunnable origin.
- **Notebooks are for exploring; scripts are for keeping.** *Jupyter notebooks* (started with `jupyter lab`) mix code, output, and notes — unbeatable for looking around, dangerous for results, because cells run in any order and silently hold stale state. The rule: the moment an analysis produces something you care about, promote it from notebook to a script that runs top to bottom from a clean start. If it only works in the notebook, it does not work.
- **Seed your randomness.** Any script using random draws (like our bootstrap) should fix the generator's *seed* — `rng = numpy.random.default_rng(42)` — so reruns give identical numbers. Reproducible randomness sounds like an oxymoron and is actually the foundation of trustworthy simulation.
- **Write down what you did.** A dated line — what you ran, what came out — turns next month's "why does this table disagree with the old one?" from archaeology into lookup. Version control (next) automates most of it.

7.4 Git in ten commands

Git is a version-control system: it snapshots your project so any past state can be recovered, compared, or shared. For solo research its value is blunt: **results you cannot reproduce do not exist**, and a result is reproducible when you can name the exact code state that made it. Install from git-scm.com; these ten commands are a complete working vocabulary:

#	Command	What it does
1	<code>git init</code>	turn the current folder into a repository (once per project)
2	<code>git status</code>	show what changed since the last snapshot
3	<code>git add -A</code>	stage all changes for the next snapshot
4	<code>git commit -m "message"</code>	take the snapshot, labeled with a message
5	<code>git log --oneline</code>	list all snapshots, newest first
6	<code>git diff</code>	show exactly what you edited since staging/last commit
7	<code>git restore <file></code>	throw away uncommitted changes to a file
8	<code>git switch -c <name></code>	start a side branch to try something without risk
9	<code>git clone <url></code>	copy a repository (yours or anyone's) to a machine
10	<code>git push</code>	upload your snapshots to a remote host (e.g. GitHub)

The working loop is $2 \rightarrow 3 \rightarrow 4$, a few times per session, with messages a stranger could follow ("add 200d base-rate table, n printed per cell"). Two refinements finish the setup: a `.gitignore` file listing what git should never track (the `.venv/` folder; large raw data files — the *code that fetches* data is tracked, the gigabytes are not), and — once you have a free GitHub account — `git push` as your offsite backup. From this point on, "which code produced this figure?" always has an exact answer: the *commit* — the snapshot — whose label says so. The backtesting paper builds its reproducibility protocol directly on this habit.

8 A first end-to-end analysis

Everything above, in one sitting: pull the S&P 500, compute returns, measure rolling volatility, build a no-look-ahead conditional base-rate table with n printed per cell and the $n < 15$ flag enforced, and draw a chart. This is the one place in the library where complete beginner code belongs in the body; save it as `scripts/first_analysis.py` in the Section 7.3 layout and run it with the environment active. (The figure scripts of this paper deliberately use a leaner no-pandas style for minimal dependencies; this listing is the pandas idiom you will actually write daily. Same logic, same discipline, two dialects.)

```

"""First end-to-end analysis: SPX returns, rolling vol, base rates."""
import yfinance as yf
import matplotlib.pyplot as plt

# ---- 1. Load daily S&P 500 history (cache it: raw data is read-only) ----
spx = yf.Ticker("^GSPC").history(start="2005-01-01")
spx.to_csv("data/raw/spx_daily.csv")          # snapshot what you fetched

# ---- 2. Daily returns -----
spx["ret"] = spx["Close"].pct_change()        # simple daily return
spx["up"] = spx["ret"] > 0                    # outcome: did the day close up?

# ---- 3. Rolling 21-day volatility, annualized -----
spx["vol21"] = spx["ret"].rolling(21).std() * (252 ** 0.5)

# ---- 4. Conditions known at YESTERDAY's close (no look-ahead!) -----
spx["ma200"] = spx["Close"].rolling(200).mean()
spx["prior_up"] = spx["up"].shift(1)          # yesterday's sign
spx["above_ma"] = (spx["Close"] > spx["ma200"]).shift(1) # yesterday vs MA

df = spx.dropna(subset=["ret", "prior_up", "above_ma"])

# ---- 5. Conditional base-rate table, n printed for every cell -----
table = (df.groupby(["prior_up", "above_ma"])["up"]
        .agg(p_up="mean", n="count"))
print("P(up day) unconditional: "
      f"{df['up'].mean():.3f}  (n={len(df)})")
print(table.round(3))

small = table[table["n"] < 15]
if len(small):
    print("\nWARNING - cells below n=15 (stories, not statistics):")
    print(small.round(3))

# ---- 6. Chart: price, trend line, and the market's temperature -----
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(9, 6), sharex=True)
ax1.plot(spx.index, spx["Close"], lw=0.8, label="S&P 500")
ax1.plot(spx.index, spx["ma200"], lw=0.8, label="200-day average")
ax1.set_ylabel("index level"); ax1.legend(frameon=False)
ax2.plot(spx.index, spx["vol21"], lw=0.8, color="tab:orange")
ax2.set_ylabel("21-day vol, annualized")
fig.suptitle("First analysis: price, trend, volatility")
fig.tight_layout()
fig.savefig("figures/first_analysis.png", dpi=150)
print("\nwrote figures/first_analysis.png")

```

Walk through what each block practices. Block 1 fetches *and immediately caches* (Section 7.2). Block 2 turns prices into returns — the object all statistics in this paper live on. Block 3 is Figure 3's volatility clustering made visible in your own chart: the `.rolling(21).std()` is a moving one-month standard deviation, annualized by the square-root convention of Section 3.2; its plot will

show the calm plateaus and the storm spikes of 2008-like and 2020-like episodes. Block 4 is the discipline that separates analysis from self-deception: both conditions are computed, then **shifted by one day**, so that day t is classified only by what was knowable at $t-1$ — delete those two `.shift(1)` calls and you have manufactured look-ahead bias in the most common way it occurs in the wild. Block 5 is Section 5's table on your own machine — with the `n` column and the `n < 15` tripwire coded in, so the discipline runs even when you forget to. Block 6 draws the chart and saves it to `figures/`, where a script-produced artifact belongs.

Run it, then read your own output against this paper: your unconditional $P(\text{up})$ should land near the low-to-mid 0.5s; your conditional cells should cluster unheroically near it (Section 5.2's honest flatness); your vol panel should show clustering. Then make it yours — different start date, a different index, a third condition — and watch what happens to the `n`'s as you refine. That last experience, cells thinning as conditions stack, teaches Section 5.3 better than any paragraph can. When an experiment produces something worth keeping: commit (Section 7.4), and you have completed one full cycle of the workflow every later paper in this library assumes.

9 Common misconceptions

- **"A 70% win rate means a good strategy."** The central fallacy of Section 3.5. Win rate is one of *two* factors; payoff is the other, and expectancy — their product-sum — is the only number that decides. Figure 2's 70%-winner loses 0.10R per trade forever. Corollary: high-hit-rate strategies *feel* best and are the easiest vehicles for slow ruin.- **"Returns are basically bell-curved; the crashes are freak one-offs."** Figure 1: 189 beyond- 4σ days against 1.6 expected, excess kurtosis 17.3, across a century. The extremes are not exceptions to the distribution; they *are* the distribution, arriving on their own schedule. Any risk logic that starts "assuming normality" has assumed away the part that ends accounts.- **"The market has been up 8 of the last 10 times this happened — that's an edge."** An `n = 10` cell is a story (Section 5.3); the bootstrap interval on 8-of-10 is roughly "anything from a coin flip up." Ask for `n`, ask for the base rate, ask for the interval — in that order — before "edge" enters the sentence.- **"More history is always better statistics."** Only under stationarity (Section 4.3). A 90-year average blends machines that no longer exist; a 60-session window is a story with a trend line. Honest practice shows both windows with both `n`'s, and lets neither silently stand in for the other (Section 5.4).
- **"It went up right after X happened, so X was the reason."** Correlation plus a narrative is not causation (Section 4.1): common causes (VIX and stocks), reverse causation, and searched-for coincidences (Bangladeshi butter) all produce exactly this experience. The test is whether the story predicted the sign *in advance* — and whether the sign of the data, checked, actually matches the story.- **"Six months of live profits proves my system."**

Section 6, lesson 1: 20% of paths of a *mathematically losing* strategy were profitable after 50 trades, and even 250 trades left the loser's confidence interval touching zero. Live profit over small n is weak evidence — encouraging, worth continuing to measure, and years short of proof.- **"Volatility is unpredictable — it's all random anyway."** Backwards, in an instructive way: *direction* is the nearly-unpredictable part (lag-1 autocorrelation -0.11); *volatility* is strongly persistent ($+0.34$, positive for 20 straight lags). Confusing the two forfeits the one forecasting problem where the data is actually on your side (Section 4.2).- **"I'll clean up the workflow once I find something that works."** By the time something "works," an unhygienic workflow cannot tell you whether the finding is real or an artifact of look-ahead, stale notebook state, or an overwritten input file (Section 7.3). Hygiene is not the reward for finding an edge; it is the instrument that detects one.

10 Glossary

About forty terms, alphabetical; italics at first use in the text correspond to entries here.

- **Annualization** — scaling a statistic to a yearly basis; daily volatility is annualized by multiplying by $\sqrt{252}$ (trading days per year).
- **Autocorrelation** — the correlation of a series with a time-shifted copy of itself; measures whether a series "remembers" its own past.
- **Base rate** — the plain, unconditional frequency of an event in a sample; the bar any conditional claim must beat.
- **Base rate neglect** — the psychological bias of judging conditional claims without asking the unconditional frequency first.
- **Bootstrap** — estimating the uncertainty of a statistic by recomputing it on many resampled versions (drawn with replacement) of the original data.
- **Cache** — a locally saved copy of downloaded data, so analyses rerun offline and stay stable when the source changes.
- **Commit** — a labeled git snapshot of a project's state; the exact code identity behind a reproducible result.
- **Conditional probability** — the frequency of an outcome among only those cases where a stated condition held; written $P(\text{outcome} \mid \text{condition})$.
- **Confidence interval** — the range of values for the true quantity that is plausibly consistent with your sample; wide interval = weak evidence.
- **Correlation** — a number in $-1 \dots +1$ measuring how two series move together; describes co-movement, never causation by itself.

- **Distribution** — the full description of a random quantity: which values occur and how often; the "machine" behind the data.
- **Drawdown** — the decline from a portfolio's previous peak to a subsequent trough; the lived experience of risk.
- **Expectancy** — average profit per trade: $\text{hit rate} \times \text{average win} - \text{loss rate} \times \text{average loss}$; the number that decides profitability.
- **Fat tails** — a distribution's property of producing extreme outcomes far more often than the normal curve predicts.
- **Histogram** — a bar chart of how many observations fall in each bin of values; the basic picture of a sample's distribution.
- **Hit rate (win rate)** — the fraction of trades that close profitable; meaningless without the payoff (see expectancy).
- **In-sample / out-of-sample** — data used to build or select an idea versus data held back to test it; the heart of the backtesting paper.
- **Kurtosis (excess)** — a measure of tail weight relative to the normal distribution (which scores 0); S&P 500 daily returns score ≈ 17 .
- **Lag** — the time offset in autocorrelation; lag 1 compares each day with the following day.
- **Look-ahead bias** — using information in an analysis that was not yet available at the moment being analyzed; the cardinal backtesting sin.
- **Mean** — the ordinary average; sensitive to every point, and dominated by extremes under fat tails.
- **Median** — the middle value of sorted data; robust to outliers; divergence from the mean signals skewed or fat-tailed data.
- **Moving average** — the average of the last k observations, recomputed each day (e.g. the 200-day average of closes); a slow trend measure.
- **Non-stationarity** — failure of stationarity: the data-generating machine changes over time, as markets structurally do.
- **Normal distribution** — the symmetric, thin-tailed bell curve, fully set by mean and standard deviation; a poor model for market tails.
- **Outlier** — an observation far from the bulk of the data; in returns, usually the most economically important observations.
- **Payoff (ratio)** — average win size relative to average loss size; the partner of hit rate inside expectancy.

- **R-multiple** — a trade outcome expressed in units of the amount risked (R); $+2.5R$ = the trade made $2.5\times$ its risk.
- **Recency bias** — overweighting recent observations; under non-stationarity, partially rational — the sin is silent substitution for the long sample.
- **Regime** — a period in which the market machine's settings are roughly stable (e.g. a low-volatility regime); statistics pool across regimes.
- **Resampling** — building alternative datasets by drawing from the observed one; the mechanical core of the bootstrap.
- **Return (simple)** — the percentage change of price over a period: $(\text{new} - \text{old}) / \text{old}$; the basic object of all analysis here.
- **Seed** — the fixed starting value of a random-number generator, making simulated randomness exactly reproducible.
- **Skewness** — a measure of a distribution's asymmetry; negative = long left tail (rare, violent losses).
- **Standard deviation** — the typical distance of observations from their mean (root of the variance); in markets called volatility.
- **Standard error** — the typical amount an *estimate* would vary across alternative samples; the error bar on a computed number.
- **Stationarity** — the assumption that the distribution generating the data does not change over time; the fine print under every statistic.
- **Variance** — the average squared distance from the mean; the standard deviation squared.
- **Virtual environment** — a project-private Python package space (venv), keeping projects from breaking each other.
- **Volatility** — the standard deviation of returns; the market's "temperature."
- **Volatility clustering** — the strong persistence of volatility: big-move days follow big-move days; visible as high autocorrelation of absolute returns.
- **Z-score** — a value expressed in standard deviations from the mean; 1987-10-19 was a -17σ day against this sample's ruler.

Evidence basis

Every substantive claim in this paper rests on one of three kinds of basis, distinguished here so a reader can tell at a glance which is which — **own reproducible computation** (recomputed from free data or a self-contained model, reproducible via the cited script), **literature** (peer-reviewed article or textbook, cited inline), and **practitioner consensus** (desk practice with no clean

citation, treated as the weakest). Nothing here is called *validated* on external sources alone; the stronger word requires an own out-of-sample computation.

Claim / result	Basis
S&P 500 daily returns have fat tails: excess kurtosis 17.3 and 189 days beyond 4σ against 1.6 expected under normality (1927–2026)	Own reproducible computation — figures/fig_return_histogram.py
Heavy tails are a long-established, cross-market property of financial returns	Literature — (Mandelbrot 1963)
A 70%-win strategy can carry negative expectancy (−0.10R); expectancy, not hit rate, decides profitability	Own reproducible computation — figures/fig_expectancy.py
Return direction barely repeats (lag-1 −0.114) while absolute returns cluster for weeks (lag-1 +0.335)	Own reproducible computation — figures/fig_autocorrelation.py
Conditional next-day up-rates stay within ~3 points of the 54.3% unconditional base rate	Own reproducible computation — figures/fig_base_rates.py
Every estimate carries a bootstrap confidence interval, and small samples yield uselessly wide ones	Own reproducible computation — figures/fig_base_rates.py
Base-rate neglect and small-sample overconfidence are documented cognitive biases	Literature — (Kahneman 2011)
A correlation found by searching (butter production vs the S&P 500) is not a predicted one	Literature — (Leinweber 2007)
The $n < 15$ minimum-cell rule for conditional tables	Practitioner consensus — not independently verified
The direction check reflex, adopted after an in-house sign-inversion error	Practitioner consensus — not independently verified

References

This paper is deliberately self-contained: every number in it is computed by the accompanying scripts on free data, and the references are a short annotated *reading ladder* — published books, ordered from most accessible to most advanced — rather than an academic bibliography. (Two in-text attributions for specifics: the Bangladeshi-butter demonstration is David Leinweber's, published as "Stupid Data Miner Tricks: Overfitting the S&P 500" (*Journal of Investing*, 2007); base-rate neglect is documented in the judgment-under-uncertainty research program of Daniel Kahneman and Amos Tversky, surveyed in Kahneman's book below.)

- ****Kahneman, D. (2011). *Thinking, Fast and Slow*. New York: Farrar, Straus and Giroux.**** — The psychology under Section 5: base-rate neglect, small-sample overconfidence, and the narrative reflex that turns correlation into causation. Read first; it explains why the disciplines in this paper feel unnatural and are necessary.
- ****Aronson, D. R. (2006). *Evidence-Based Technical Analysis: Applying the Scientific Method and Statistical Inference to Trading Signals*. Hoboken, NJ: Wiley.**** — The classic book-length case that trading claims must be tested like scientific hypotheses, with extended treatments of data mining and sampling error; the natural bridge from this paper to the backtesting paper.
- ****Sinclair, E. (2013). *Volatility Trading* (2nd ed.). Hoboken, NJ: Wiley.**** — A practitioner's statistics in action: distributions, volatility measurement and forecasting, and position sizing, written by an options trader; the right next step toward this library's volatility track.
- ****López de Prado, M. (2018). *Advances in Financial Machine Learning*. Hoboken, NJ: Wiley.**** — The advanced rung: labeling, backtest overfitting, and the deflated performance statistics referenced throughout the backtesting and overfitting-control papers. Read after those; listed here so the ladder has a visible top.

The sequel to this paper is *Backtesting & Hypothesis Testing* — it assumes exactly the material built here (distributions and base rates, expectancy versus hit rate, standard errors, and the Section 7 toolchain) and turns it into a testing discipline: pre-registration, out-of-sample splits, transaction costs, and the catalogue of ways backtests deceive.

All references above are publicly accessible: all are published books available through any bookseller or library (the one cited journal article appears in a published journal). No proprietary, course, or trading-academy material is cited.